



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Sampling and Splitting

Data analysis and Data scientists often want to do the following:

1. Take a random sample of a large data set
2. Randomly Split dataset to create training, validation, and test sets.
3. Combine data frames

1. Take a random sample of a large data set

Assume you have a huge data frame of billions of records.

You are developing some logic for data analysis.

And you want to try and test your logic on the real data.

But your Dataframe is huge.

Testing your logic on billions of records will take time and resources.

So you want to test it on a small portion of the real data frame.

Spark allows you to take a random sample from your actual Dataframe and use it for testing.

2. Randomly split dataset to create training, validation and data sets.

Machine learning and data science often need to split the Dataframe into two or more parts.

They use one part of the data frame for training their models.

Other parts may be used for validation and testing the models.

So randomly splitting a Dataframe is also a common requirement.

Spark allows you randomly split your Dataframes.

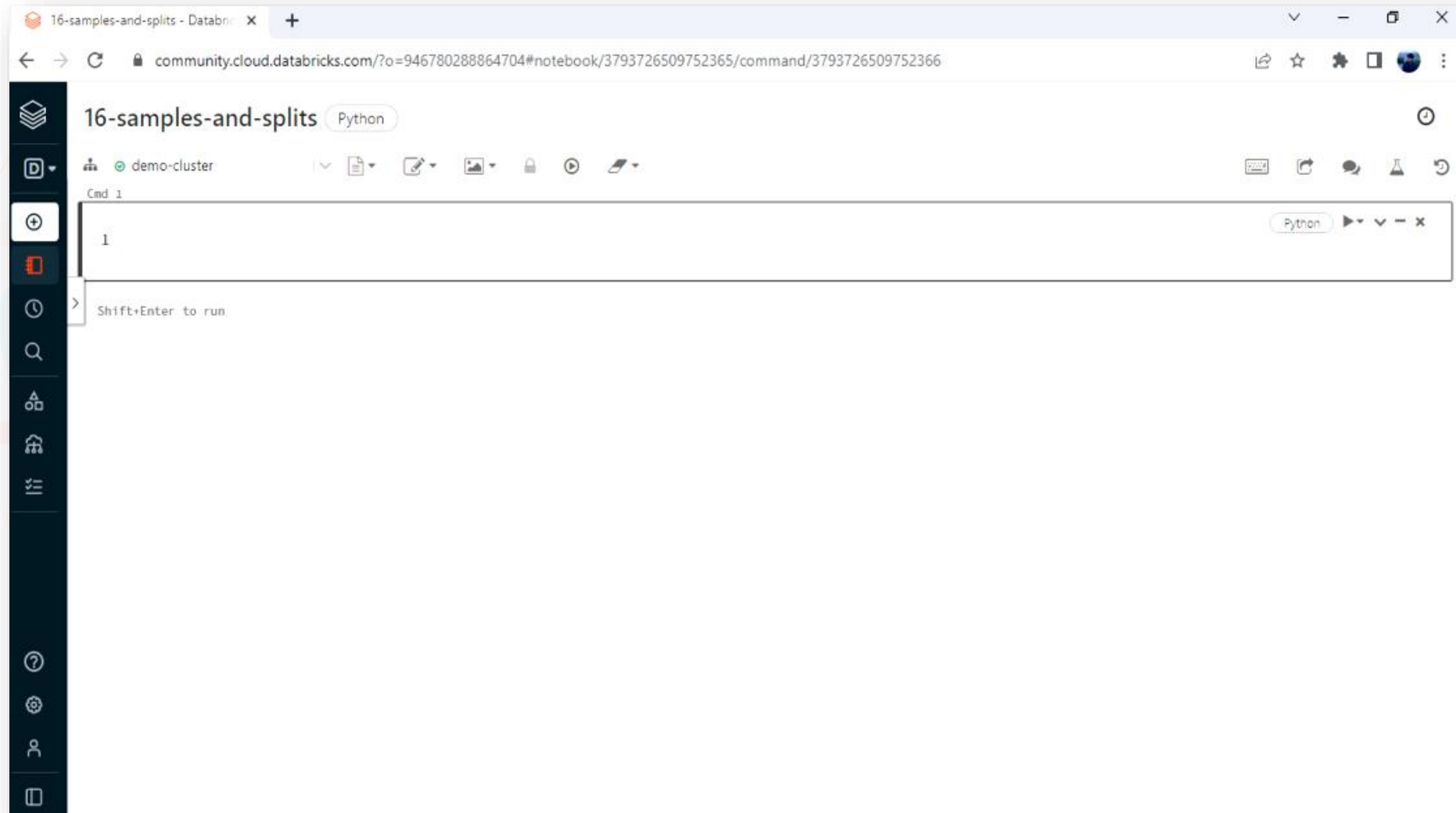
3. Combine Dataframes

When you learn to split a data frame into two or more parts, it is also essential to learn to combine two parts into a single Dataframe.

Spark offers two methods for sampling your Dataframe:

1. `sample(withReplacement=None, fraction=None, seed=None)`
2. `sampleBy(col, fractions, seed=None)`

Go to your Databricks workspace and create a new notebook.
(Reference : 16-samples-and-splits)



We need a large data frame so we can try sampling and splitting. So I have created one Dataframe using the airline sample data.

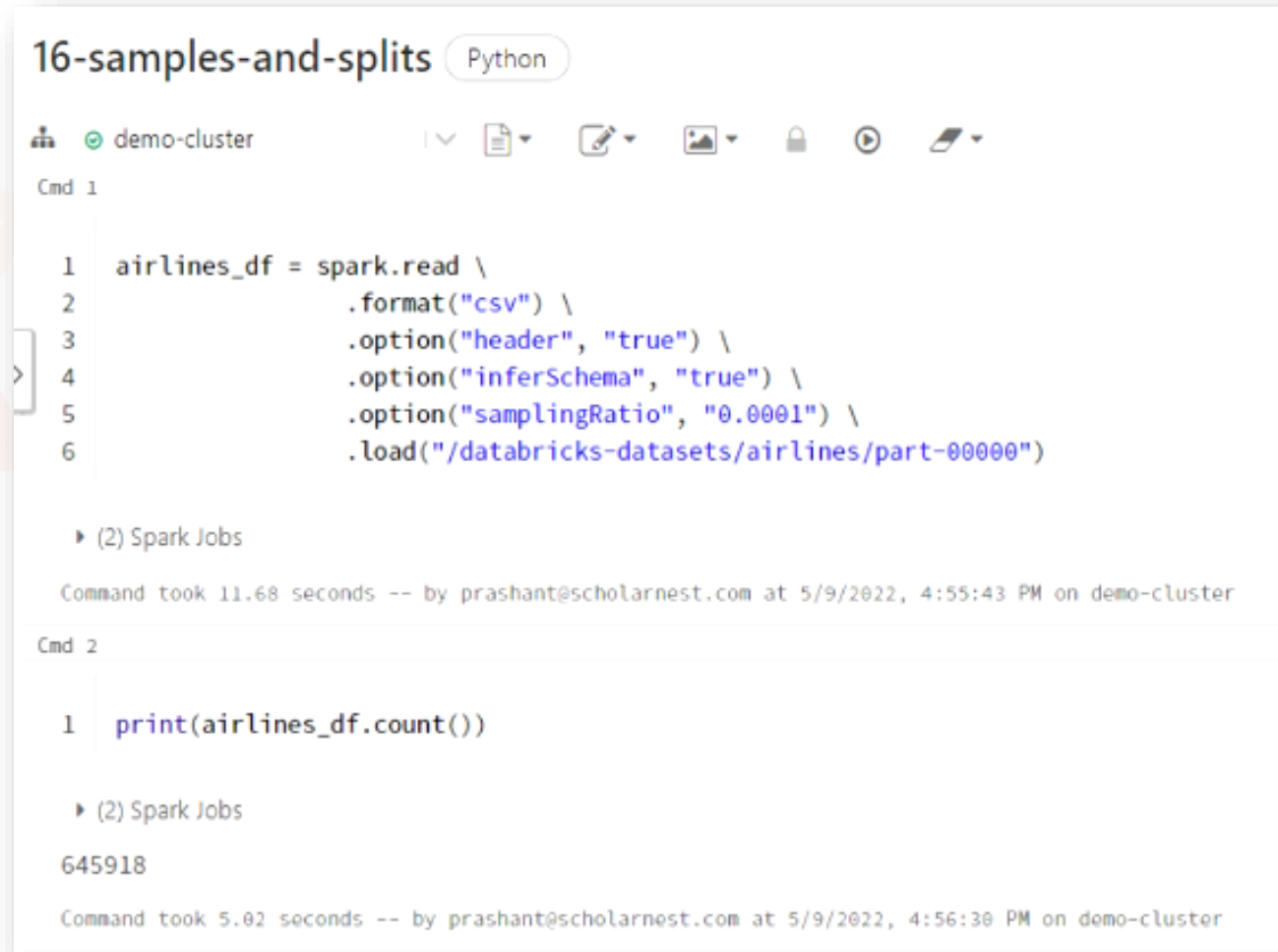


The screenshot shows a Databricks notebook interface. At the top, the notebook title is "16-samples-and-splits" and the language is set to "Python". Below the title, the cluster name "demo-cluster" is displayed. The command area shows a single command labeled "Cmd 1" which is a multi-line Python statement to read a CSV file from a Databricks dataset, applying a sampling ratio of 0.0001. The command is as follows:

```
1 airlines_df = spark.read \  
2     .format("csv") \  
3     .option("header", "true") \  
4     .option("inferSchema", "true") \  
5     .option("samplingRatio", "0.0001") \  
6     .load("/databricks-datasets/airlines/part-00000")
```

Below the code, it indicates "(2) Spark Jobs" were executed. At the bottom, a status message states: "Command took 11.68 seconds -- by prashant@scholarnest.com at 5/9/2022, 4:55:43 PM on demo-cluster".

We have approximately six lakhs and forty-five thousand records. Now I want to take 10 thousand of these records. You can do a select * and limit 10000. But that will not be a proper mix or random sampling of records. When we want to take a sample, we mean that each record in the data frame should have an equal probability of being selected in the sample.



```
16-samples-and-splits Python

demo-cluster

Cmd 1

1 airlines_df = spark.read \
2     .format("csv") \
3     .option("header", "true") \
4     .option("inferSchema", "true") \
5     .option("samplingRatio", "0.0001") \
6     .load("/databricks-datasets/airlines/part-00000")

▶ (2) Spark Jobs

Command took 11.68 seconds -- by prashant@scholarnest.com at 5/9/2022, 4:55:43 PM on demo-cluster

Cmd 2

1 print(airlines_df.count())

▶ (2) Spark Jobs

645918

Command took 5.02 seconds -- by prashant@scholarnest.com at 5/9/2022, 4:56:30 PM on demo-cluster
```

So I am sampling the *airlines_df* and creating *random_sample_df*. The `sample()` method takes three arguments.

The `fraction` is a required parameter, and it represents the sample percentage. I need 10%, so I am setting the `fraction` to 0.1. You can give any value from 0 to 1, representing the sample size in percentage.

The other two arguments are optional.

The `withReplacement()` take a true/false value. Can we?

If you want to take the same record twice, set the `withReplacement` to true. Otherwise, set it to false.

The `seed` is the seed of randomization.

You can use a seed to reproduce the same sample again.

It could be an integer value, and using the same seed across multiple runs will produce the same sample Dataframe.

```
> Cmd 3  
  
1 random_sample_df = airlines_df.sample(fraction=0.1, withReplacement=False, seed=0)
```

```
Command took 0.03 seconds -- by prashant@scholarnest.com at 5/9/2022, 4:59:29 PM on demo-cluster
```

You can see the count of the random sample shown below. The count may not be precisely 10% of your base data frame, but it will be close.

It happens because the `sample()` method does not give a guarantee to provide an exact fraction of the total count.

However, the result will be close to the fraction specified.

Cmd 3

```
1 random_sample_df = airlines_df.sample(fraction=0.1, withReplacement=False, seed=0)
```

Command took 0.03 seconds -- by prashant@scholarnest.com at 5/9/2022, 4:59:29 PM on demo-cluster

Cmd 4

```
1 random_sample_df.count()
```

▶ (2) Spark Jobs

Out[4]: 64459

Command took 3.63 seconds -- by prashant@scholarnest.com at 5/9/2022, 4:59:56 PM on demo-cluster

We have the airlines_df Dataframe shown below.

We have the UniqueCarrier field highlighted below.

Now let's assume you want to bucket your data by the UniqueCarrier and then take samples from each bucket.

Each bucket is known as strata, and the approach is called Stratification.

Sometimes, it is desired to take stratified samples.

Cmd 5

1 display(airlines_df)

(1) Spark Jobs

	Year	Month	DayofMonth	DayOfWeek	DepTime	CRSDepTime	ArrTime	CRSArrTime	UniqueCarrier	FlightNum	TailNum
1	1987	10	14	3	741	730	912	849	PS	1451	NA
2	1987	10	15	4	729	730	903	849	PS	1451	NA
3	1987	10	17	6	741	730	918	849	PS	1451	NA
4	1987	10	18	7	729	730	847	849	PS	1451	NA
5	1987	10	19	1	749	730	922	849	PS	1451	NA
6	1987	10	21	3	728	730	848	849	PS	1451	NA
7	1987	10	22	4	728	730	852	849	PS	1451	NA

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)



Command took 1.07 seconds. -- by prashant@scholarnest.com at 5/9/2022, 5:00:44 PM on demo-cluster

So I am filtering airlines_df and taking only three UniqueCarrier codes. This step is not necessary for generating stratified examples. But I am limiting the options so you can understand them easily. Then I counted records for each carrier code, and the result is shown below. So in my base_df, I have 56091 records for AA, 63104 for DL, and so on.

Now I have a requirement to create two strata. One for AA and another for DL. I do not care about the PS or other carriers because that's not in my requirement. Once we have two strata, I want to take 10K records from each stratum and create a sample of 20K records.

```
Cmd 6

1  base_df = airlines_df.filter("UniqueCarrier in ('AA', 'DL', 'PS')")
2  print(base_df.count())
3
4  base_df.groupBy("UniqueCarrier") \
5      .count() \
6      .orderBy("UniqueCarrier") \
7      .show()

► (4) Spark Jobs

147140
+-----+
|UniqueCarrier|count|
+-----+
|          AA|56091|
|          DL|63104|
|          PS|27945|
+-----+

Command took 10.52 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:04:26 PM on demo-cluster
```

Requirements:

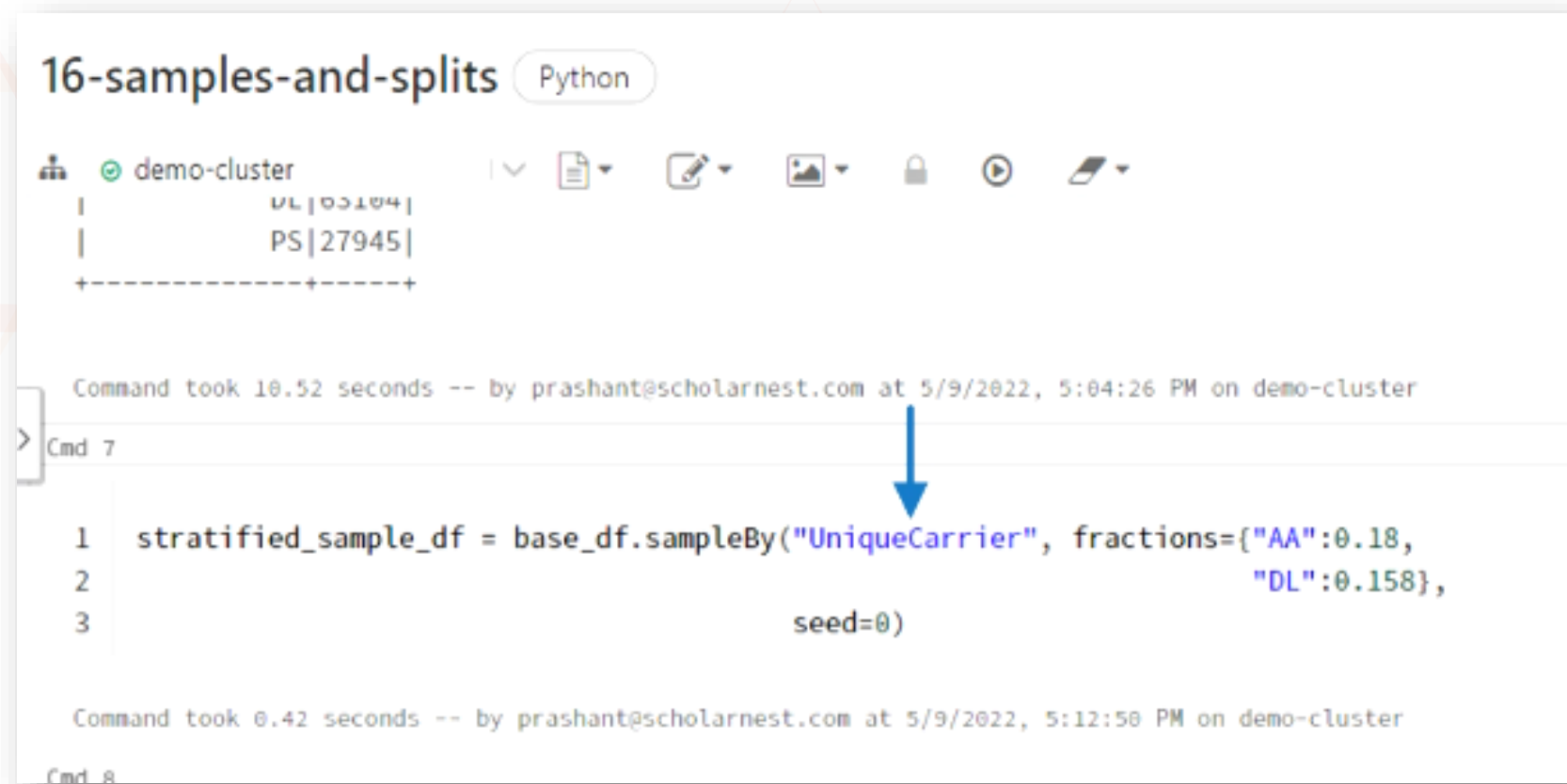
- 20K samples for AA and DL
- Random Sampling
- Each code should have approx. 10K samples

I need data from two Carrier codes. So I will create two strata. For selecting records from strata, you need a sampling fraction for the strata. And here is the calculation of sampling fraction.

Sampling Fraction Calculation:

- AA : $10,000/56091 = 0.18$
- DL : $10,000/63104 = 0.158$

I am using the `sample()` method on my base data frame to create a stratified sample. The `sample()` method takes three arguments. The first argument is the strata column name. So this example will create strata using the unique carrier codes. I have three unique carriers in my Dataframe, so the `sample()` will create three strata. A stratum is nothing but a kind of record bucket. So all the records for AA will go in one bucket. Similarly, all the records for DL will be in a different bucket, and so on. Each unique carrier code will become one bucket, and we call them strata.



The screenshot shows a Jupyter Notebook window titled "16-samples-and-splits" with a "Python" language selector. The interface includes a toolbar with icons for file operations, a terminal, and a console. Below the toolbar, a code cell is displayed with the following Python code:

```
1 stratified_sample_df = base_df.sampleBy("UniqueCarrier", fractions={"AA":0.18,  
2                                     "DL":0.158},  
3                                     seed=0)
```

A blue arrow points from the text "UniqueCarrier" in the code to the "UniqueCarrier" argument in the `sampleBy` method. The notebook also shows command execution logs, including "Command took 10.52 seconds" and "Command took 0.42 seconds".

The second argument is for strata fractions. I want to sample only AA and DL. So I will give only two fractions.

The fraction for AA is 0.18, and DL is 0.158. I am giving only two fractions. So Spark will sample records from only these two buckets.



The screenshot shows a Databricks notebook interface. At the top, the notebook title is "16-samples-and-splits" with a "Python" language selector. Below the title, there's a toolbar with icons for file operations, a lock, and a play button. The main area displays a command execution history. The first command took 10.52 seconds. The second command, labeled "Cmd 7", is a Python code snippet for stratified sampling. A blue arrow points to the "fractions" parameter in the code. The command took 0.42 seconds.

```
16-samples-and-splits Python
```

demo-cluster

VL|00104|
PS|27945|
+-----+-----+

Command took 10.52 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:04:26 PM on demo-cluster

Cmd 7

```
1 stratified_sample_df = base_df.sampleBy("UniqueCarrier", fractions={"AA":0.18,  
2                                     "DL":0.158},  
3                                     seed=0)
```

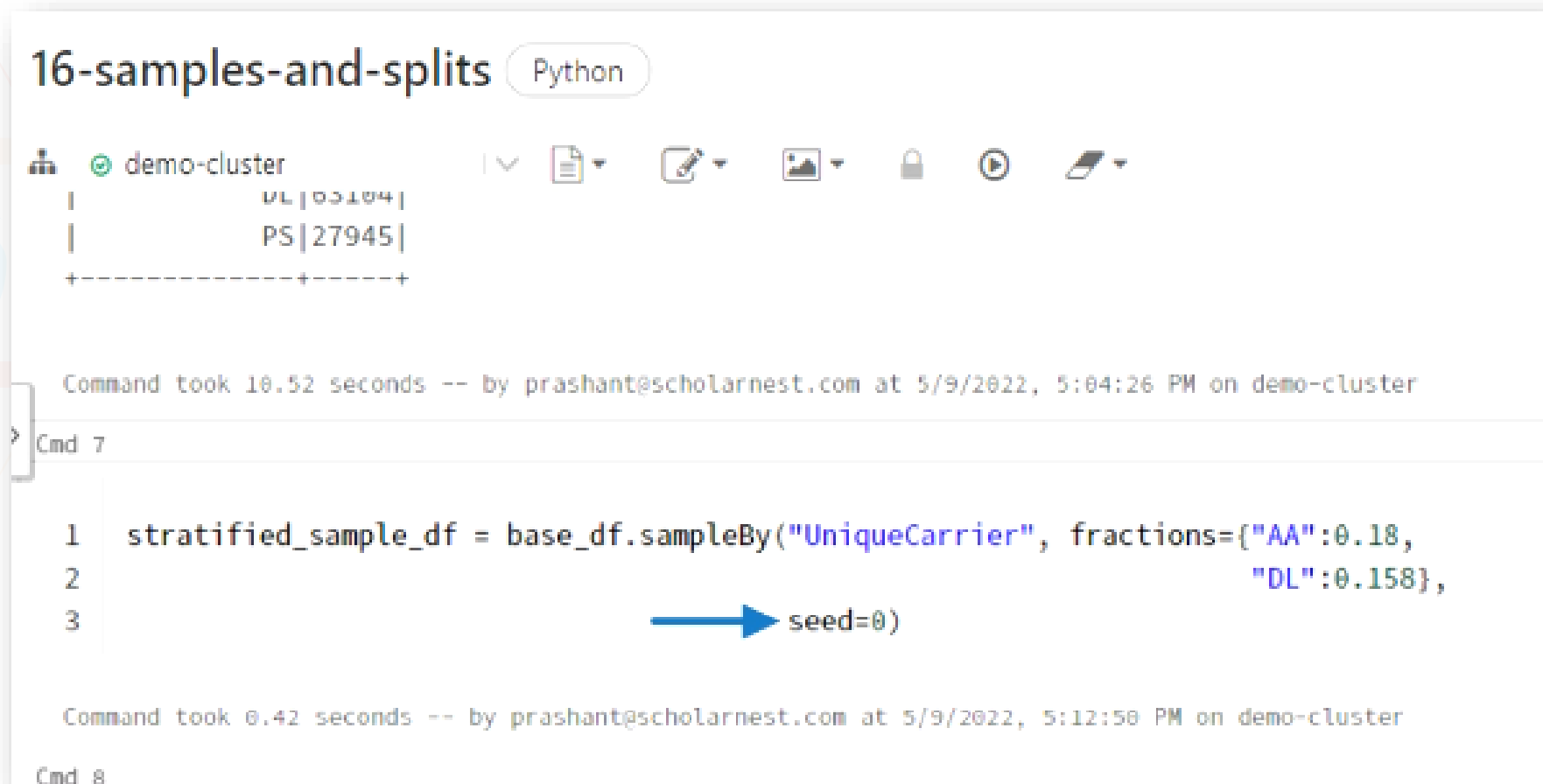
Command took 0.42 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:12:50 PM on demo-cluster

Cmd 8

The last argument is the randomization seed.

You can use a seed to reproduce the same sample again.

It could be an integer value, and using the same seed across multiple runs will produce the same sample Dataframe.



The screenshot shows a Jupyter Notebook interface. At the top, the title is "16-samples-and-splits" with a "Python" language selector. Below the title, there's a toolbar with icons for file operations, a search icon, a refresh icon, a lock icon, a play icon, and a close icon. The main area displays a command and its output. The command is: `stratified_sample_df = base_df.sampleBy("UniqueCarrier", fractions={"AA":0.18, "DL":0.158}, seed=0)`. The output shows the command took 0.42 seconds. A blue arrow points to the `seed=0` argument in the command.

```
16-samples-and-splits Python
```

demo-cluster

DL | 03104 |
PS | 27945 |
+-----+

Command took 10.52 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:04:26 PM on demo-cluster

Cmd 7

```
1 stratified_sample_df = base_df.sampleBy("UniqueCarrier", fractions={"AA":0.18,  
2                                     "DL":0.158},  
3                                     → seed=0)
```

Command took 0.42 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:12:50 PM on demo-cluster

Cmd 8

You can see the stratified Dataframe shown below, we got the desired output.
That's all about sampling data frames.

We have two options.

1. sample() method, which applies simple random sampling.
2. We also have a sampleBy() that applies stratified random sampling.

```
Cmd 8

1 stratified_sample_df.groupBy("UniqueCarrier") \
2     .count() \
3     .orderBy("UniqueCarrier") \
4     .show()

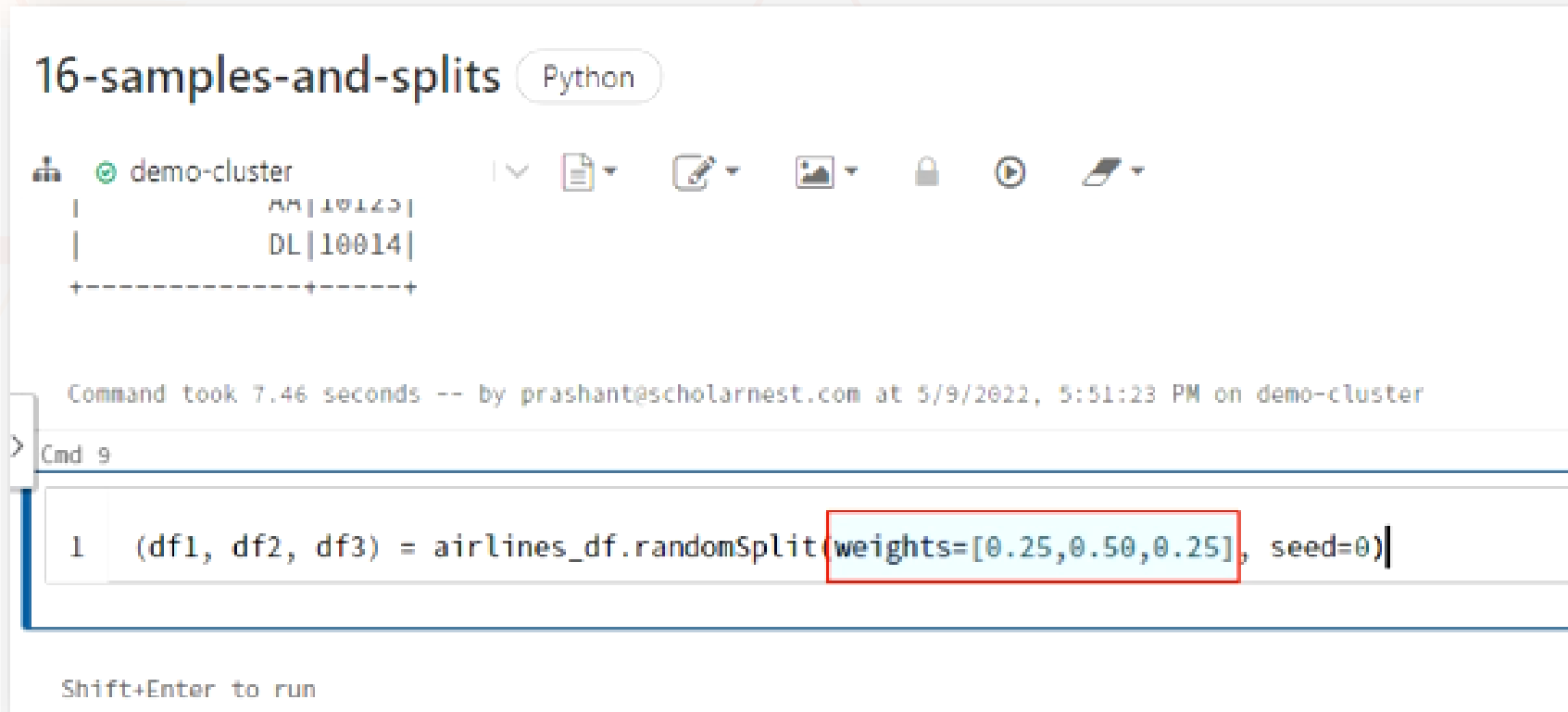
▶ (2) Spark Jobs

+-----+
|UniqueCarrier|count|
+-----+
|          AA|10123|
|          DL|10014|
+-----+
```

Command took 7.46 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:51:23 PM on demo-cluster

Now let's move onto the next topic of Splitting a Dataframe.

Let's assume you have `airlines_df`. And you want to split this Dataframe into three parts. The split must be random. And the split ratio should be 1:2:1. This ratio is the same as 25%:50%:25%. Spark gives you the `randomSplit()` method for splitting a dataframe. So I can use the `randomSplit()` method on the `airlines_df` and split it into multiple parts as shown below. I am splitting it into three parts. But you can split and create 2 or 5 or whatever number. The `randomSplit()` takes two arguments. The first one is a list of weights for each split. You can give weights in floating-point numbers. The weight must be a floating-point number, and the spark will normalize the values into a split percentage.



The screenshot shows a Jupyter Notebook interface. At the top, the title is "16-samples-and-splits" with a "Python" language selector. Below the title, there's a toolbar with icons for saving, running, and other actions. The main area displays a code cell with the following content:

```
1 (df1, df2, df3) = airlines_df.randomSplit(weights=[0.25,0.50,0.25], seed=0)
```

The `weights=[0.25,0.50,0.25]` part of the code is highlighted with a red box. Below the code cell, there's a status bar that says "Shift+Enter to run".

You can check the count of the three data frames as shown below.

The counts are in the given percentage. It may not be precisely 25%:50%:25%, but it should be close enough.

The randomization doesn't give a guarantee for an exact count.



The screenshot shows a terminal window with two commands. Command 9 splits a DataFrame into three parts with weights [0.25, 0.50, 0.25]. Command 10 prints the row counts of the three resulting DataFrames. The output shows 161240, 323047, and 161631 rows respectively. A blue arrow points to the output of the third DataFrame. The terminal also shows the execution time and user for each command.

```
> Cmd 9  
  
1 (df1, df2, df3) = airlines_df.randomSplit(weights=[0.25,0.50,0.25], seed=0)  
  
Command took 0.05 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:54:25 PM on demo-cluster  
  
Cmd 10  
  
1 print(df1.count(), df2.count(), df3.count())  
  
▶ (6) Spark Jobs  
161240 323047 161631  
  
Command took 29.04 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:55:05 PM on demo-cluster
```

The next topic is to combine Dataframes.

Can we combine smaller Dataframe to create one large Dataframe? Yes. We can if they have the same schema.

Spark allows you to combine two or more data frames as long as they have the same schema.

we have a union method to combine two data frames. If you have three data frames to combine, you can create a chain of union() methods and combine multiple data frames. And you can see that the Dataframe df4's count is equal to the sum of the count of Dataframes df1 and df3.



The screenshot shows a Jupyter Notebook interface with two code cells. The first cell, labeled 'Cmd 11', contains the code `df4 = df1.union(df3)`. A blue arrow points from the text 'Command took 29.04 seconds' above it to the `union` method. The second cell, labeled 'Cmd 12', contains the code `df4.count()`. Below this code, the output is displayed as `Out[12]: 322871`. The notebook also shows execution times for each command: 29.04 seconds for Cmd 11 and 0.07 seconds for Cmd 12. A third execution time of 17.39 seconds is shown at the bottom of the screenshot.

```
Command took 29.04 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:55:05 PM on demo-cluster  
Cmd 11  
1 df4 = df1.union(df3)  
  
Command took 0.07 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:56:17 PM on demo-cluster  
Cmd 12  
1 df4.count()  
  
▶ (2) Spark Jobs  
Out[12]: 322871  
  
Command took 17.39 seconds -- by prashant@scholarnest.com at 5/9/2022, 5:56:37 PM on demo-cluster
```

You should remember a few critical things about the union:

1. Both the combining Dataframe must have the same schema.
2. Union is applied by column order. So the first column of df2 combines with the first column of df1. So the column order is essential.
3. Union method is the same as UNION ALL in SQL. So it will not remove duplicates. If you have the same record in two data frames, the combined Dataframe will have both records. If you want to eliminate duplicates, you can use the `distinct()` method on the combined Dataframe.



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com